Check for updates

Networks

WILEY

RESEARCH ARTICLE

Speeding-Up Graph Algorithms via Clique Partitioning

Akshar Chavan[1] | Sanaz Rabinia[2] | Daniel Grosu[2] | Marco Brocanelli[1]

[1]Department of Electrical and Computer Engineering, The Ohio State University, Columbus, OH, USA | [2]Department of Computer Science, Wayne State University, Detroit, MI, USA

Correspondence: Daniel Grosu (dgrosu@wayne.edu)

Received: 18 August 2025 | **Revised:** 5 January 2026 | **Accepted:** 20 March 2026

Keywords: clique partitioning | graph algorithms | speeding-up graph algorithms

ABSTRACT

Reducing the running time of graph algorithms is vital for tackling real-world problems such as shortest paths and matching in large-scale graphs, where path information plays a crucial role. To address this critical challenge, this paper introduces a graph restructuring algorithm that identifies bipartite cliques and replaces them with tripartite graphs. This restructuring leads to fewer edges while preserving complete graph path information, enabling the direct application of algorithms like matching and all-pairs shortest paths to achieve significant runtime reductions, especially for large, dense graphs. The running time of the proposed algorithm for a graph $G(V, E)$, with $|V| = n$ and $|E| = m$ is $O(mn^{\delta})$, which is better than $O(mn^{\delta}\log^2 n)$, the running time of the best existing algorithm for speeding-up other graph algorithms (the Feder–Motwani (FM) algorithm), where $0 \leq \delta \leq 1$. Both the FM algorithm and the proposed algorithm are originally formulated for bipartite graphs, but can also be applied to general directed or undirected graphs. Our extensive experimental analysis demonstrates that the proposed algorithm achieves up to 21.26% higher reduction in the number of edges and runs up to 105.18× faster than the FM algorithm. On large synthetic graphs with up to 1.05 billion edges, it attains a reduction in the number of edges of up to 74.36%. On real-world graphs, it achieves a reduction in the number of edges by up to 46.8%. Furthermore, when used as a preprocessing step, our approach yields up to a 2.07× speedup for the matching algorithms on large synthetic graphs, and up to a 1.74× speedup for the All-Pairs Shortest Path algorithms on real-world graphs, when compared to using the given graph as input.

1 | Introduction

Graph algorithms are essential for analyzing complex real-world networks across diverse domains like social interactions, biological systems, and communication infrastructures. However, as these networks grow in size and complexity, the time required to process them using graph algorithms becomes excessively high. This paper tackles the critical issue of high computation time in large-size graphs by introducing a graph restructuring approach that speeds up the execution of graph algorithms such as matching and all pairs shortest paths. This approach involves transforming the graph into a more computationally suitable structure, preserving its path information, where path information refers to the existence of paths between vertices. Specifically, by restructuring the graph to reduce the number of edges, we aim to speeding-up graph algorithms. A key requirement for this is the preservation of complete path information during the graph restructuring process. Many crucial graph algorithms, such as matching and all-pairs shortest paths (APSP), rely on this global connectivity information to produce correct results.

However, to effectively speed up the downstream algorithms, the restructuring algorithm itself must be efficient and the resulting structure readily usable. Specifically, it must meet two critical conditions: (1) the graph restructuring algorithm must have a low running time to ensure that its use as a preprocessing step leads

© 2026 Wiley Periodicals LLC.

TABLE 1 | Overview of graph restructuring methods.

Technique	Goal	Mechanism	Applications	Limitations for Speeding-up Graph Algorithms
Sparsification [2–4]	Reduce # of Edges	Edge sub-sampling or filtering	Graph Algorithms, Visualization	Accuracy loss, potential disconnection
Coarsening [5, 6]	Reduce # of Vertices	Vertex grouping/aggregation	Multilevel Methods, GNN Scaling	Loss of fine-grained detail, interpolation error
Hierarchical Decomposition [7–10]	Reduce Problem Complexity	Subdivide problem using graph structure	Optimization, Modeling	Applicability depends on problem structure
Lossless Compression [7, 11–15]	Reduce Storage Space	Efficient encoding of graph data	Storage, Querying Compressed Data	Query overhead, less effective on irregular graphs
Clique Partitioning [1], **CPGR** [this paper]	Speeding-up Graph Algorithms	Extracting bipartite cliques	Graph Algorithms, Storage	—

to an overall reduction in execution time compared to running the algorithms on the original graph; and (2) the restructured graph should be directly usable as input to these algorithms or require minimal modifications. These conditions highlight the inherent challenges in designing effective graph restructuring algorithms for speeding-up graph algorithms. In this paper, we address these challenges and propose a new Clique Partitioning based Graph Restructuring (CPGR) algorithm that restructures the graph to reduce the number of edges while preserving the path information. We theoretically and experimentally show that our algorithm achieves faster running time and greater reduction in the number of edges compared to the algorithm proposed by Feder and Motwani [1], that is, the best existing algorithm used to speeding-up the execution of graph algorithms.

1.1 | Related Work

Graph-structured data are common across domains such as social networks, biology, and the web, often reaching billions of edges. To address the computational challenges posed by such scale, graph restructuring techniques have emerged as a key approach for improving the efficiency of graph algorithms through structural transformation. These restructuring methods aim to produce graph representations that are enabling fast computation. These approaches include sparsification, graph coarsening, hierarchical decomposition, and lossless compression, each offering trade-offs between speed and accuracy as shown in Table 1.

Graph sparsification reduces the edge count while approximately preserving properties like cuts, distances, or spectral characteristics, enabling faster execution of algorithms such as PageRank, at the cost of precision [2–4]. Graph coarsening merges nodes and edges to create a smaller approximation of the original graph, commonly used in multilevel algorithms for partitioning and clustering [5, 6]. While this improves scalability, it can obscure fine-grained structural details. Hierarchical decomposition organizes graphs into nested clusters or layers, facilitating divide-and-conquer approaches in tasks like community detection and multiscale visualization [7–10]. The main drawback lies in the overhead of maintaining and traversing hierarchical structures. Lossless compression techniques aim to reduce the graph size while preserving exact recoverability, typically using encoding, redundancy elimination, or succinct data structures [7, 11–15]. Although accurate, such methods can add significant overhead.

Despite their advantages, many of these techniques either compromise path-preservation or introduce substantial preprocessing overhead, limiting their applicability to algorithms that require full connectivity information such as matching and APSP. A more targeted approach is path-preserving graph restructuring, proposed by Feder and Motwani [1], that partitions the graph into bipartite cliques to speeding-up downstream graph algorithms such as bipartite matching, APSP, and vertex connectivity. Their approach guarantees path preservation but suffers from sequential dependencies during clique extraction, which may limit the performance in terms of execution time and reduction in the number of edges.

Building on this foundation, our work proposes a novel Clique Partitioning-based Graph Restructuring (CPGR) algorithm. Unlike Feder and Motwani's iterative singleton clique extraction, CPGR introduces multiple and larger clique extraction, enabling more efficient restructuring while preserving exact path information. This makes CPGR a promising candidate for preprocessing large-scale graphs to accelerate a wide range of graph algorithms.

2 | Background and Motivation

2.1 | Background

Feder and Motwani [1] proposed a graph restructuring algorithm—referred to as FM in this paper—designed for bipartite graphs $G(U, W, E)$, where $|U| = |W| = n$ and $|E| = m$. The algorithm identifies complete bipartite subgraphs called *δ-cliques* with left and right partitions of size $\lceil n^{1-\delta} \rceil$ and $k(n, m, \delta) = \left\lfloor \frac{\delta \log n}{\log(2n^2/m)} \right\rfloor$, respectively, for some constant $0 \leq \delta \leq 1$. Each δ-clique is then replaced by a more compact tripartite graph that introduces a new vertex z_q, connecting it to all vertices in the clique's left and right partitions. This transformation reduces the number of edges from $|U_q| \times |W_q|$ to $|U_q| + |W_q|$ per clique,

10970037, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/net.70038 by Wayne State University Library, Wiley Online Library on [22/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

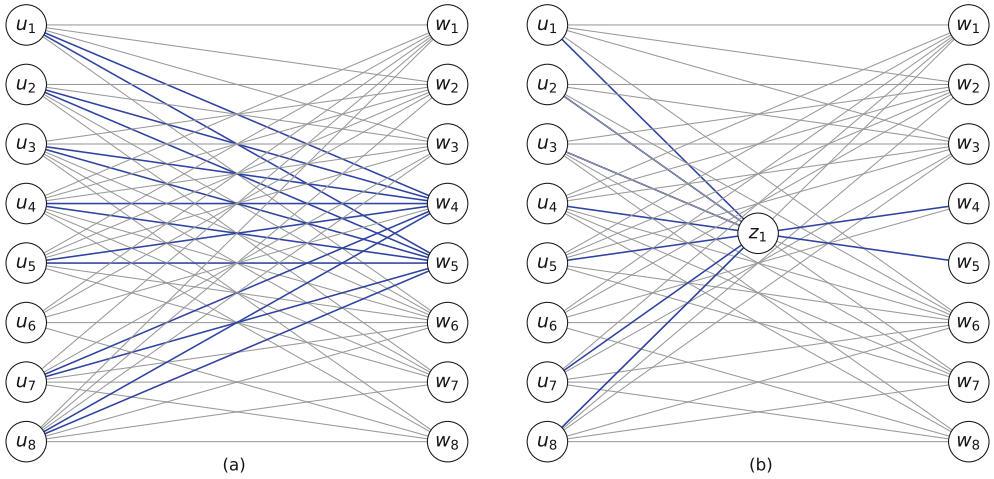


FIGURE 1 | (a) Given bipartite graph $G(U, W, E)$ and (b) the tripartite graph that replaces the δ-clique with left partition $\{u_1, u_2, u_3, u_4, u_5, u_7, u_8\}$ and right partition $\{w_4, w_5\}$ in the restructured graph $G^*(U, W, Z, E^*)$.

achieving significant reduction in the number of edges while preserving the original graph's connectivity between U and W. For instance, Figures 1a and 1b show an example of a δ-clique with left partition $\{u_1, u_2, u_3, u_4, u_5, u_7, u_8\}$ and right partition $\{w_4, w_5\}$ in a bipartite graph G, and the corresponding tripartite graph that replaces it in the restructured graph G^*, respectively. The number of edges in the δ-clique is $14 = 7 \times 2$ while the number of edges in the corresponding tripartite graph is $9 = 7 + 2$, thus the number of edges in the restructured graph is reduced by 5. Thus, the restructured graph $G^* = (U, W, Z, E^*)$ consists of three disjoint vertex sets, where Z contains one new vertex per extracted clique, and edges in E^* connect vertices across these partitions. Although designed for bipartite graphs, the approach can be extended to general graphs as well. Further details on the FM algorithm, including the clique extraction process and the use of neighborhood trees, are provided in Appendix A.

2.2 | Motivation

We motivate our approach by highlighting three critical aspects related to extracting and replacing δ-cliques that influence both the running time and the reduction in the number of edges. These aspects arise in existing approaches and suggest opportunities for improving how dense substructures are identified and processed.

Vertex selection. A key challenge in identifying δ-cliques for reducing the number of edges lies in selecting vertices for the right partition W_q that share a large set of common neighbors in U. In approaches such as the FM algorithm, vertices for the right partition are selected using a combinatorial function that accounts for vertex degrees across different levels of a neighborhood tree. At each level of the neighborhood tree, vertex degrees within the corresponding partitions are aggregated to determine which branch to explore. As a result, vertex selection is driven by combined degree contributions across partitions rather than by the individual potential of vertices to form dense bipartite subgraphs with U, which may cause vertices with higher degree to be overlooked when forming dense δ-cliques.

Extracting δ-cliques. The process of forming δ-cliques relies on selecting $\hat{k} = k(n, \hat{m}, \delta)$ vertices from the right partition W, where $\hat{m}$ is the number of remaining edges in the graph and n is the number of vertices in each partition. As the algorithm progresses and more cliques are extracted, the number of edges $\hat{m}$ decreases, which in turn affects the value of $\hat{k}$. Due to the stepwise nature of the function $k(n, \hat{m}, \delta)$, the value of $\hat{k}$ often remains constant across multiple iterations. As illustrated in Figure 2a, this results in a staircase-like progression in which several iterations operate under the same $\hat{k}$ while removing relatively few edges per iteration. This behavior suggests that the iterative structure may not fully exploit the potential of each $\hat{k}$ value, particularly during phases in which $\hat{k}$ remains unchanged over multiple iterations.

Update mechanism. After selecting vertices from the right partition, the manner and timing of degree updates play a crucial role in determining which edges remain available for future δ-clique discovery. In the existing approach (FM algorithm), degree updates are applied incrementally as candidate vertices are selected, rather than after a δ-clique has been fully determined. As a result, degree reductions may be applied to vertices based on partial clique construction, rather than solely on edges that are ultimately removed by an extracted clique. Such early updates can reduce the availability of edges that could participate in subsequent δ-cliques, particularly in sparse or moderately dense graphs. This observation highlights the importance of update mechanisms that accurately reflect only the edges eliminated by an extracted clique.

2.3 | Our Contributions

This paper introduces a novel algorithm, the Clique Partitioning-based Graph Restructuring (CPGR) algorithm. Our main contributions are summarized as follows:

1. We present CPGR, a graph restructuring algorithm that removes multiple δ-cliques within a single iteration, achieving a running time of $O(mn^\delta)$. This improves upon the $O(mn^\delta \log^2 n)$ running time of the FM algorithm.

10970037, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/net.70038 by Wayne State University Library, Wiley Online Library on [22/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

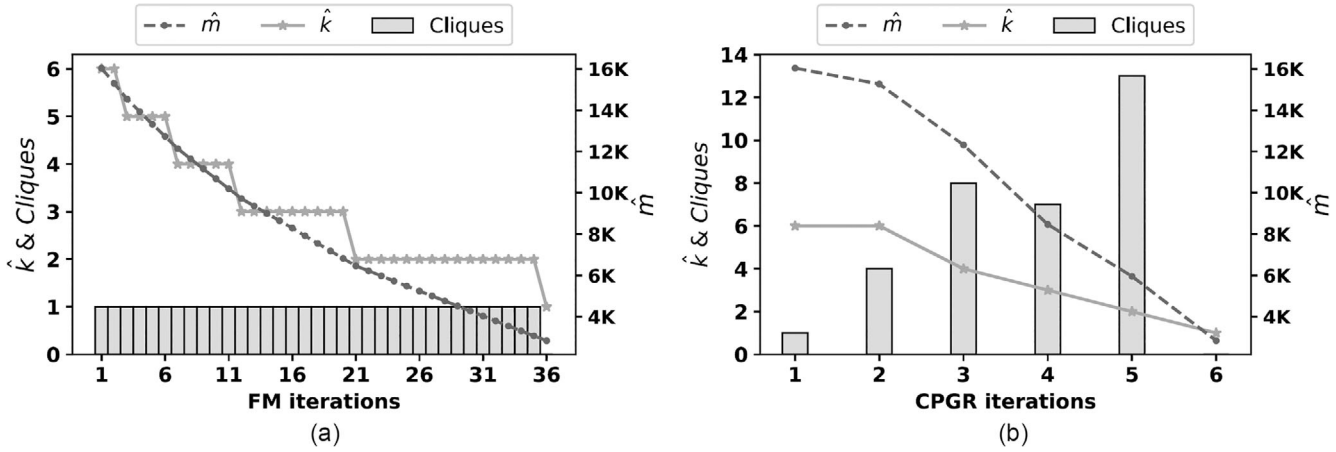


FIGURE 2 | Progression of $\hat{m}$, $\hat{k}$, and number of cliques extracted for a graph with 128 vertices in each bi-partition, density 0.98, and $\delta = 1$ by (a) FM and (b) CPGR.

2. CPGR allows vertices to participate in multiple clique extractions during the restructuring process. For a fixed $\hat{k}$, the algorithm is designed to extract as many valid δ cliques as possible in each iteration, which increases the total number of extracted cliques, particularly in large and high density graphs, and leads to a larger number of edges removed per iteration, as illustrated in Figure 2b.
3. CPGR achieves an average reduction in the number of edges that is *at least* as good as the FMalgorithm.
4. Experimental analysis show that CPGR achieves up to 21% higher reduction in the number of edges and runs up to 105.18× faster than the FM algorithm.
5. Experimentally, we show the practical utility of CPGRas a preprocessing step, achieving a speedup of up to 2.07× for matching and 1.74× for APSP on different graph datasets.

3 | The Proposed Algorithm

3.1 | Clique Partitioning-Based Graph Restructuring (CPGR) Algorithm

This section details the design of our Clique Partitioning-based Graph Restructuring (CPGR) algorithm, presented in Algorithm 1. For a given input graph G, CPGR obtains a restructured graph G^* of G by iteratively finding bipartite cliques (i.e., complete bipartite subgraphs) in $G(U, W, E)$ and replacing them with corresponding tripartite graphs until finding new bipartite cliques does not contribute to the reduction in the number of edges of G. The size of the right partition of the cliques is determined by $\hat{k} = k(n, \hat{m}, \delta)$, which guarantees a δ-clique, and the selection of vertices $w_j \in W$ depends on the degree of vertices $d_{w_j} = |N(w_j)|$, where $N(w_j)$ denotes the set of neighbors of vertex w_j. The partition of G into bipartite cliques guarantees that each edge in G is in exactly one clique, that is, $E(C_i) \cap E(C_j) = \emptyset$, $\forall i, j$ and $i \neq j$, where $E(C_i)$ is the set of edges of δ-clique C_i. The input of CPGR consists of the adjacency matrix A of G, and a constant δ.

ALGORITHM 1 | CPGR: clique partitioning-based graph restructuring algorithm.

Input: A: Adjacency matrix;
δ: a constant such that $0 \leq \delta \leq 1$.
1: $q \leftarrow 0$
2: $n \leftarrow |W|$
3: $\hat{m} \leftarrow |E|$
4: $d_w \leftarrow [0]_n$
5: $\mathcal{C} \leftarrow \emptyset$
6: **for** $i = 1, \ldots, n$ **do**
7: **for** $j = 1, \ldots, n$ **do**
8: $d_{w_j} \leftarrow d_{w_j} + E_{u_i, w_j}$
9: $\hat{k} \leftarrow \left\lfloor \frac{\delta \log n}{\log(2n^2/\hat{m})} \right\rfloor$
10: **while** $\hat{k}$>1 **do**
11: $(\mathscr{C}, \hat{A}, \hat{d_w}) \leftarrow \mathsf{CSA}(q, \hat{k}, A, n, d_w)$
12: $\mathcal{C} \leftarrow \mathcal{C} \cup \mathscr{C}$
13: $d_w \leftarrow \hat{d}_w$
14: $A \leftarrow \hat{A}$
15: $\hat{m} \leftarrow \sum_{j=1}^{|W|} d_{w_j}$
16: $q \leftarrow q + |\mathscr{C}|$
17: $\hat{k} \leftarrow \left\lfloor \frac{\delta \log n}{\log(2n^2/\hat{m})} \right\rfloor$
18: $Z \leftarrow \emptyset$; $E^* \leftarrow \emptyset$; $\mathcal{E}_C \leftarrow \emptyset$
19: **for** $i \in \mathcal{C} = \{\mathscr{C}_1(U_1, W_1), \ldots, \mathscr{C}_q(U_q, W_q)\}$ **do**
20: $Z \leftarrow Z \cup \{z_i\}$
21: $E^* \leftarrow E^* \cup \{(u, z_i), (z_i, w)\} \quad \forall u \in U_i, \quad \forall w \in W_i$
22: $\mathcal{E}_C \leftarrow \mathcal{E}_C \cup E_i$
23: $E^* \leftarrow E^* \cup \{(u, w) | (u, w) \in E \setminus \mathcal{E}_C\}$
24: **Output:** $G^*(U, Z, W, E^*)$, the restructured graph of G.

Initialization (lines 1–5). For the given bipartite graph $G(U, W, E)$, CPGR initializes the index q of the bipartite cliques extracted from G, the number of vertices, n where $n = |U| = |W|$, the number of edges $\hat{m}$, and the degree of the vertices $d_{w_j} \forall j = 1, \ldots, |W|$, initialized to $[0]_n$, the zero vector of size n. CPGR also initializes the set of extracted bipartite cliques $\mathcal{C}$, to the empty set.

Partition size (lines 6–9). CPGR computes the degree of each vertex $w_j \in W$, $d_{w_j} = |N(w_j)|$, where E_{u_i, w_j} is 1, if there is an

10970037, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/net.70038 by Wayne State University Library, Wiley Online Library on [22/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

edge between u_i and w_j, and 0, otherwise (lines 6–8). It determines the size of the right partition of the bipartite clique, $\hat{k}$ (line 9), which guarantees the existence of a δ-clique with $k(n, \hat{m}, \delta)$ [1]. It is important to say that $\hat{m} = |E| = \sum_{j=1}^{|W|} d_{w_j}$.

Clique extraction (lines 10–17). CPGR proceeds with extracting δ-cliques until extracting new cliques does not contribute to the reduction in the number of edges. This happens when $\hat{k} = 1$, which results in obtaining trivial bipartite cliques. Thus, the while loop in lines 10 to 17 is executed until trivial cliques are produced.

Clique Stripping Algorithm (CSA), presented in Algorithm 2, finds δ-cliques in the bipartite graph G. CSA takes the updated index q of the bipartite clique, the size $\hat{k}$ of the right partition of the bipartite clique, the adjacency matrix A, the size n of the left partition of G, and the degree of vertices $d_{w_j} = |N(w_j)|,\ \forall w_j \in W$, as input. CSA, which is described in Section 3.2, returns the set $\mathscr{C}$ of bipartite cliques that it determined in the current execution, the updated adjacency matrix $\hat{A}$, and the updated degree of vertices $\hat{d_w}$. In line 12, CPGR adds the new bipartite cliques to the set of all bipartite cliques C, updates the degree of vertices d_w (line 13), and adjacency matrix A (line 14). In line 15, CPGR updates the number of edges, $\hat{m}$ in the given graph $G(U, W, E)$ after extracting the δ-cliques, and the clique index q (line 16). Finally, in line 17, after the δ-cliques are removed, CPGR updates $\hat{k}$ for the next iterations.

Graph restructuring (lines 18–23). When the while loop (lines 10–17) terminates, CPGR restructures the graph by adding a new vertex set Z to the bipartite graph. Each vertex in the left and right partitions of clique $\mathscr{C}_q$ is connected to a new vertex z_q, reducing the number of edges from $|U_q| \times |W_q|$ to $|U_q| + |W_q|$ (lines 18–23). In line 22, $\mathcal{E}_C$ is updated with edges from the extracted δ-cliques. Remaining edges in the bipartite graph $G(U, W, E)$, not part of any bipartite δ-cliques, are connected directly in the tripartite graph (line 23). In line 24 the algorithm returns the restructured graph $G^*(U, Z, W, E^*)$ having a reduced number of edges.

In Appendix A we provide an example showing the execution of CPGR.

3.2 | Clique Stripping Algorithm (CSA)

The Clique Stripping Algorithm (CSA), given in Algorithm 2, extracts δ-cliques from the given bipartite graph $G(U, W, E)$. CSA initially selects the vertices for the right partition K_c of the clique C_c and then the common neighbors for the set K_c. The input of CSA consists of the updated index q of the clique, the size $\hat{k}$ of the right partition which guarantees a δ-clique in G, the adjacency matrix A, the number of vertices n, and the degree of vertices $d_{w_j} = |N(w_j)|$.

Initialization (lines 1–3). CSA initializes the set $\mathcal{K}$ of selected vertices for clique extraction to the empty set. In line 2, it sorts d_w in non-increasing order of the degrees of vertices $w_j \in W$, that is, $d_{w_{\pi(1)}} \geq d_{w_{\pi(2)}} \geq \ldots \geq d_{w_{\pi(n)}}$. In line 3 it initializes the sorted vertices list index j to 1.

ALGORITHM 2 | CSA: clique stripping algorithm.

Input: q: Index of bipartite clique;
$\hat{k}$: Size of the right bipartition in bipartite clique;
A: Adjacency matrix;
n: Number of vertices in left/right bipartition of G;
d_w: Degree of vertices in bipartition W.

1: $\mathcal{K} \leftarrow \emptyset$
2: Sort d_w in non-increasing order. Let $d_{w_{\pi(1)}}, d_{w_{\pi(2)}}, \ldots, d_{w_{\pi(n)}}$ be the order.
3: $j \leftarrow 1$
4: **while** $d_{w_{\pi(j)}} \geq d_{w_{\pi(\hat{k})}}$ **do**
5: $\quad \mathcal{K} \leftarrow \mathcal{K} \cup \{w_{\pi(j)}\}$
6: $\quad j \leftarrow j + 1$
7: $\gamma \leftarrow \left\lfloor \frac{|\mathcal{K}|}{\hat{k}} \right\rfloor$
8: **for** $c = q + 1, \ldots, q + \gamma$ **do**
9: $\quad K_c \leftarrow \{w_{\pi_{(j)}} \in \mathcal{K} \mid (c - (q+1)) \cdot \hat{k} < j \leq (c - q) \cdot \hat{k}\}$
10: $\quad U_{K_c} \leftarrow \{u_i \in U \mid K_c \subseteq N(u_i)\}$
11: $\quad C_c \leftarrow \{(U_{K_c}, K_c)\}$
12: $\quad$ Update A, by removing edges $(U_{K_c} \times K_c) \in C_c$
13: $\quad$ Update d_w by subtracting $|U_{K_c}|$ from each $d_{w_{\pi(j)}}$, where $w_{\pi(j)} \in K_c$
14: $\mathscr{C} \leftarrow \{C_{q+1}, \ldots, C_{q+\gamma}\}$
15: **Output** $(\mathscr{C}, \hat{A}, \hat{d_w})$

Vertex selection (lines 4–7). CSA selects vertices for clique extraction based on their degrees, since vertices with higher degree are more likely to share a larger set of common neighbors and thus participate in dense bipartite cliques. Rather than selecting exactly $\hat{k}$ vertices, CSA constructs a candidate set $\mathcal{K} = \{w_{\pi(j)} | d_{w_{\pi(j)}} \geq d_{w_{\pi(\hat{k})}}\}$, which contains all vertices whose degree is at least the $\hat{k}$th largest degree in the sorted sequence d_w. As a result, $|\mathcal{K}| \geq \hat{k}$ in general. CSA adds these vertices to $\mathcal{K}$ by iterating through the sorted order and incrementing the index j (lines 5–6). Thus, the while-loop in lines 4–6 ensures that all vertices tied at the $\hat{k}$th degree threshold are included, avoiding arbitrary tie-breaking and forming a maximal candidate set for the right partition.

This threshold-based selection increases the likelihood of identifying a larger set of common neighbors in the left partition, thereby improving the quality of the extracted δ-cliques. In line 7, CSA computes $\gamma = \lfloor |\mathcal{K}|/\hat{k} \rfloor$, which determines how many δ-cliques can be extracted. The set $\mathcal{K}$ is then partitioned into γ subsets of size $\hat{k}$ (line 9), allowing CSA to extract multiple δ-cliques in a single execution.

Clique extraction (lines 8–13). The for loop (lines 8–9) first partitions $\mathcal{K}$ into subsets K_c, such that $\bigcup_{c=q+1,\ldots,q+\gamma} K_c = \mathcal{K}$ and $K_{q+i} \bigcap K_{q+j} = \emptyset,\ i = 1, \ldots, \gamma, j = 1, \ldots, \gamma, i \neq j$. Then it constructs U_{K_c} such that each vertex in U_{K_c} is part of the set of common neighbors of K_c (line 10). In line 11, CSA forms clique C_c with partitions U_{K_c} and K_c, where $|U_{K_c}| \geq \lceil n^{1-\delta} \rceil$ and $|K_c| = \hat{k}$. After forming the clique C_c, in line 12, the algorithm updates the adjacency matrix by removing the edges in the clique C_c and in line 13, updates the degrees of the vertices that are part of the clique by removing the size of left partition from the degree of each vertex $w_{\pi(j)} \in K_c$, that is, $d_{w_{\pi(j)}} = d_{w_{\pi(j)}} - |U_{K_c}|,\ \forall\ w_{\pi(j)} \in$

10970037, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/net.70038 by Wayne State University Library, Wiley Online Library on [22/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

K_c, $c = q+1, \ldots q+\gamma$. Finally, in line 14, CSA forms $\mathscr{C}$, the set of all the cliques extracted in the current execution, and in line 15, it returns the set of cliques $\mathscr{C}$, the updated adjacency matrix $\hat{A}$, and the updated degrees of vertices $\hat{d}_w$ to CPGR.

Time complexity of CSA. In line 2 CSA takes $O(n \log n)$ to sort the degrees of vertices d_w. The while loop in lines 4–6, takes at most $O(n)$ to select vertices for the δ-cliques. The for loop (lines 8–13) executes $O(\gamma)$ times, where $1 \leq \gamma \leq \frac{n}{\hat{k}}$. In the for loop, line 9 and 10 takes $O(\hat{k})$ and $O(n\hat{k})$ to find the right and left partition of the cth δ-clique, respectively. CSA takes $O(n\hat{k})$ to remove the edges in the cth δ-clique, in line 12. It takes $O(\hat{k})$ to update the degrees $d_w \in K_c$, in line 13. Therefore, the total running time of CSA is dominated by lines 10 and 12 and thus CSA takes $O(n\gamma\hat{k})$ time.

Time complexity of CPGR. CPGR takes $O(n^2)$ to calculate the degrees d_w of vertices in W in lines 6–8. The running time of the while loop (lines 10–17) is dominated by the running time of the function CSA in line 11 which is given by Algorithm 2. CSA takes $O(n\gamma\hat{k})$ to remove $\gamma\hat{k}n^{1-\delta}$ edges. Therefore, on average it takes $O(n^{\delta})$ time to remove one edge from the given graph. Thus, the while loop in CPGR takes $O(mn^{\delta})$ time to extract δ-cliques. In the end, to restructure the graph, CPGR in lines 18–21, takes linear time. Therefore, the total running time of CPGR is $O(mn^{\delta})$ which is dominated by the while loop in lines 10–17.

4 | Properties of CPGR

The transformation performed by CPGR preserves vertex-to-vertex reachability and the connected components of the original graph. Consequently, the existence of paths between any pair of vertices in G is maintained in the restructured graph G^*. However, CPGR does not necessarily preserve local structural properties such as vertex degrees, neighborhood structure, or exact shortest-path distances, since edges belonging to δ-cliques are replaced by paths of length two. We formalize this preservation of path information in the following theorem.

Theorem 1. *The restructured graph $G^*(U, W, Z, E^*)$ obtained by CPGR preserves the path information (i.e., vertex-to-vertex reachability) of the original graph $G(U, W, E)$.*

Proof. Let's assume that CPGR extracts only one δ-clique C_q from the given graph G. In the δ-clique C_q, the right partition $K_q \subseteq W$ is formed by selecting $\hat{k}$ vertices from W and the left partition $U_q \subseteq U$ is formed with the common neighbors of K_q, forming a complete bipartite graph with edge set E_q. The restructured graph G^* is formed with the same left and right partitions U and W of the graph G, and a third partition Z which is a set of additional vertices associated with each of the δ-cliques that CPGR extracts. Our main concern is with the edges $(u_i, w_j) \in E^*$ in the restructured graph G^*. E^* contains the edges that replace the δ-clique C_q by adding the additional vertex $z_q \in Z$, and the set of edges $\hat{E} = E \setminus E_q$ which were not part of the δ-clique C_q. It can be easily seen that $\hat{E} \subset E$, that is, the edges in $\hat{E}$ are edges in G, and the remaining edges $E^* \setminus \hat{E}$ in E^* are the edges that replace C_q in G^*. Each edge $(u_i, w_j) \in E_q$, is replaced by two edges, (u_i, z_q) and (z_q, w_j), where $u_i \in U$, $w_j \in W$, and $z_q \in Z$. Therefore, for each edge $(u_i, w_j) \in E_q$, there exists a path from u_i to w_j composed of two edges, (u_i, z_q) and (z_q, w_j), that passes through the additional vertex $z_q \in Z$, thus preserving the path information. This holds true for all the δ-cliques extracted by CPGR. □

In the following, we determine an upper bound on the number of edges in the restructured graph obtained by CPGR. First, we state a theorem from [1] that guarantees the existence of a δ-clique in a bipartite graph. This theorem is necessary in the proof of the bound.

Theorem 2 ([1, theorem 2.2]). *Every bipartite graph $G(U, W, E)$ contains a δ-clique.*

Lemma 1. *Given a graph $G(U, W, E)$, where $n = |U| = |W|$, $m = |E|$, and a constant δ, $0 < \delta < 1$, if $m < 2n^{2-\frac{\delta}{2}}$ then extracting δ-cliques and replacing them with tripartite graphs as done in CPGR does not lead to a reduction in the number of edges of G.*

Proof. In the δ-clique based graph restructuring performed by CPGR, the restructured graph $G^*(U, W, Z, E^*)$ is obtained from G by replacing $|E_q| = |U_q \times K_q|$ edges in a δ-clique C_q with $|E^*| = |U_q| + |K_q|$ edges in G^* and adding an additional vertex $z_q \in Z$ for each extracted δ-clique C_q. The size of the right partition of C_q is $|K_q| = k = k(n, m, \delta) = \left\lfloor \frac{\delta \log n}{\log(2n^2/m)} \right\rfloor$. If $m < 2n^{2-\frac{\delta}{2}}$ then the size of the right partition $|K_q|$ is $k = 1$. Therefore, the number of edges in the δ-clique is $|E_q| = |U_q| \cdot 1 = |U_q|$. Those edges are replaced by $|E^*| = |U_q| + \hat{k} = |U_q| + 1$ edges in G^*. Thus, replacing the δ-clique in G with $m < 2n^{2-\frac{\delta}{2}}$, the number of edges in G^* would actually increase by 1, which does not lead to a restructuring of G. □

Theorem 3. *Let $G(U, W, E)$ be any bipartite graph with $|U| = |W| = n$ and $|E| = m > 2n^{2-\frac{\delta}{2}}$, where δ is a constant such that $0 < \delta < 1$. Then, the number of edges in the restructured graph $G^*(U, W, Z, E^*)$ obtained by CPGR is $|E^*| = O\left(\frac{m}{k(n,m,\delta)}\right)$.*

Proof. We follow the basic idea of the proof from theorem 2.4 in [1] and extend it to apply to the CPGR algorithm. We assume that initially the restructured graph G^* has 0 edges, that is, $|E^*| = 0$, and that edges are added to E^* as the algorithm progress. To estimate the number of edges in E^*, we divide the iterations of CPGR into stages, where each stage i consists of extracting one or more δ-cliques with a fixed $\hat{k}_i$. Therefore, the ith stage includes all δ-cliques extracted after the number of edges in G becomes less than $m/2^{i-1}$ for the first time, and before the number of edges in G becomes less than $m/2^i$ for the first time. For stage i, $\hat{k}$ is always going to be at least $k_i = \frac{\delta \log n}{2\log(2^i \cdot 2n^2/m)}$.

Assume that a clique C_q with right partition $K_q \subseteq W$ and left partition $U_{K_q} \subseteq U$ is extracted in stage i. Extracting C_q removes $|K_q \times U_{K_q}|$ edges from G and adds $|K_q| + |U_{K_q}|$ edges in E^*. Therefore, the average number of edges added in E^* for each edge removed from G by extracting C_q is $\rho = \frac{|K_q| + |U_{K_q}|}{|K_q \times U_{K_q}|} = \frac{1}{|K_q|} + \frac{1}{|U_{K_q}|}$. From the definition of a δ-clique, during stage i we have $|K_q| = \hat{k} \geq k_i$ and $|U_{K_q}| \geq n^{1-\delta}$, and therefore, $\rho \leq \frac{1}{k_i} + \frac{1}{n^{1-\delta}}$. The total number of

10970037, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/net.70038 by Wayne State University Library, Wiley Online Library on [22/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

edges removed from G in stage i cannot be greater than $2m/2^i$, therefore, the number of edges added to E^* during stage i is less than or equal to $\rho\frac{2m}{2^i} = \left(\frac{1}{k_i} + \frac{1}{n^{1-\delta}}\right)\frac{2m}{2^i}$.

CPGR terminates extracting cliques when the number of remaining edges in G is less than $2n^{2-\frac{\delta}{2}}$. This is to eliminate the extraction of trivial cliques, as shown in Lemma 1. Next, we determine an upper bound on the total number of edges m^* added by CPGR to G^* before it terminates extracting δ-cliques (i.e., when $m < 2n^{2-\frac{\delta}{2}}$). Therefore,

$$m^* \leq \sum_{i=1}^{\left\lceil \log\left(\frac{m}{2n^{2-\delta/2}}\right)\right\rceil} \left(\frac{1}{k_i} + \frac{1}{n^{1-\delta}}\right)\frac{2m}{2^i} \leq \sum_{i=1}^{\infty}\left(\frac{1}{k_i} + \frac{1}{n^{1-\delta}}\right)\frac{2m}{2^i}. \quad (1)$$

Since $1/(2k_i) \leq 1/k + i/(\delta \log n)$, we obtain,

$$\begin{aligned} m^* &\leq \frac{4m}{k}\sum_{i=1}^{\infty}\frac{1}{2^i} + \frac{4m}{\delta \log n}\sum_{i=1}^{\infty}\frac{i}{2^i} + \frac{2m}{n^{1-\delta}}\sum_{i=1}^{\infty}\frac{1}{2^i} \\ &\leq 2m\left(\frac{2}{k} + \frac{4}{\delta \log n} + \frac{1}{n^{1-\delta}}\right). \end{aligned} \quad (2)$$

Thus, $m^* = O\left(\frac{m}{k}\right)$. After CPGR finishes extracting cliques, there are $2n^{2-\delta/2}$ edges remaining in G. Those remaining edges are trivial cliques (i.e., single edges) that are added to G^*. The number of remaining edges $2n^{2-\delta/2}$ is in $O\left(\frac{m}{k}\right)$. Therefore, the number of edges in the restructured graph $G^*(U, W, Z, E^*)$ obtained by CPGR is $|E^*| = O\left(\frac{m}{k}\right)$. □

Extension to non-bipartite graphs. CPGR can be extended to restructure non-bipartite graphs using a similar technique to that described in [1]. Consider a directed graph $H(V, E)$ with n vertices and m edges. Initially, we transform H into a bipartite graph $G(L, R, E')$, where each vertex $v \in V$ is duplicated into $v_L \in L$ and $v_R \in R$. For each directed edge (u, v) in E, we create a directed edge (u_L, v_R) in E', with the direction from L to R. Furthermore, for each vertex $v \in V$ we add a directed edge from v_R to v_L. The proposed graph restructuring algorithm is then applied to the bipartite graph G, which includes only the edges from L to R. Once the restructured graph is computed we add to it the n directed edges from u_R to u_L, corresponding to each vertex $u \in V$. Since the path information of the original graph is preserved by this transformation, it allows different graph algorithms to work on the restructured graph as well. Undirected general graphs can be transformed first into directed graphs by simply replacing each undirected edge (u, v) in the original graph by two directed edges (u, v) and (v, u), and then applying the technique presented above to transform the directed graph into a bipartite graph. The time complexity and the compression ratio of the proposed algorithms are the same as in the case of the bipartite graphs. The time complexity and the reduced number of edges of CPGR are the same as in the case of the bipartite graphs. A similar approach for converting a general graph to a bipartite graph and then restructuring the obtained bipartite graph was employed in [12, 14].

5 | Experimental Results

We evaluated the running time and the *reduction in the number of edges*, R of CPGR by comparing it with the FM algorithm [1]. Let $m = |E|$ be the number of edges in the given graph G and $m^* = |E^*|$ be the number of edges in the restructured graph G^* and $m^* \leq m$, then we define the reduction in the number of edge as $R = m - m^*$. Due to the lack of an existing implementation, we implemented FM in C. However, its vertex selection calculations encountered machine representation limits on large graphs, restricting our comparative analysis to small bipartite graphs (up to $n = 128$ vertices per partition, $\sim$ 16k edges). To ensure a fair runtime comparison, we excluded trivial clique extraction ($\hat{k} = 1$) for both algorithms (CPGR inherently does this as per Algorithm 1, line 10). We further assessed CPGR on large bipartite graphs (up to 1.05 billion edges) and real-world datasets from the SuiteSparse Matrix Collection [16]. Additionally, we tested the utility of our restructured graph by applying the Dinitz algorithm for maximum bipartite matching to large bipartite graphs and the APSP algorithm on real-world datasets. All algorithms, were implemented in C and tested on a Linux system (AMD EPYC-74F3, 3.2 GHz, 1 core, 128 GB RAM) using GCC 8.5.0. The implementation is available at GitHub [17].

Our experiments involved synthetic bipartite graphs generated using adapted $G(n, p)$ random graph model with $|U| = |W| = n$, $n \in [32, 32\,768]$, densities $p \in [0.8, 0.98]$, up to 1.05 billion edges and general graphs with standard $G(n, p)$ with $n = 32\,000$, densities $p \in [0.4, 0.7]$, $\sim$ 409 to 716 million edges, with 10 instances per configuration. We also used real-world sparse graphs from SuiteSparse [16]. For each instance, we ran CPGR with six δ values [0.5 to 1] and show average results and standard deviations for runtime and reduction in number of edges R, as well as the performance of the application algorithms on the restructured graphs.

5.1 | Results for Small Bipartite Graphs

In this section, we compare the performance of CPGR against FM on small bipartite graphs, particularly, with $n = 2^i$ vertices in each bipartition, where i ranges from 5 to 7 and the number of edges ranging from 819 to 16 056. However, due to the limitations of FM as previously mentioned, our comparison is constrained to $n = 2^7$ and $m = 16\,056$. We compare the performance of the two algorithms using two metrics, speedup, and R relative to FM.

Reduction in number of edges R relative to FM. We define the *edge reduction ratio relative to* FM as the ratio of the R achieved by CPGR to that of FM. Figure 3a–c illustrate the relative R of CPGR compared to FM for small bipartite graphs across different δ values (0.5 to 1) and densities (0.80, 0.90, 0.98). In all these cases, the relative R ranges from 1 to 1.27, indicating that CPGR achieves at least the same, and often a greater reduction than FM. This improved performance of CPGR stems from its ability to extract multiple δ-cliques before $\hat{k}$ decreases (as shown in Figure 2b), allowing for the removal of more edges at higher $\hat{k}$ values (Section 3.2). Notably, for a graph with $n = 32$, $m = 819$, a density of 0.80, and $\delta = 0.5$, CPGR achieves an R equivalent to FM. This is because the small size of the graph limits the number of iterations required for δ-clique extraction, thus not providing CPGR with a significant advantage over FM. However, for the same $n = 32$ and δ, increasing the graph density leads to a higher R for CPGR. For instance, when m increases from 921 to 1003, the relative R rises from 1.03 to 1.25. In contrast, for a graph $n = 128$,

10970037, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/net.70038 by Wayne State University Library, Wiley Online Library on [22/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

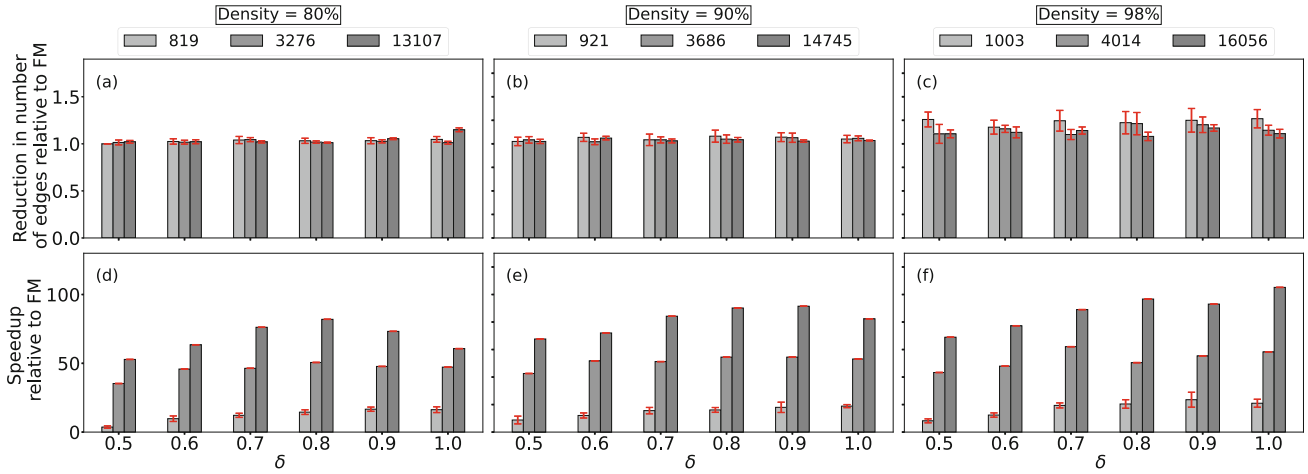


FIGURE 3 | CPGR vs. FM: Reduction in number of edges relative to FM (top row: (a), (b) and (c)) and speedup relative to FM (bottom row: (d), (e) and (f)) for graphs with 819 to 16 thousand edges.

$m = 16\,056$, density = 0.98, we observe a non-monotonic trend in the relative R as δ increases. Initially, the ratio increases from 1.1 to 1.14 as δ goes from 0.5 to 0.7. However, further increasing δ to 1 results in a decrease to 1.1. This reduction is to the fact that a larger δ increases the size of the right partition K_q of a δ-clique (since $|K_q| = \hat{k}$ and $\hat{k} \propto \delta$), consequently decreasing the likelihood of finding common neighbors.

It is important to recognize that the optimal R is influenced by the interplay between n, m, and δ. Identifying the specific combinations of these parameters that yield the highest reduction in the number of edges R remains a subject for future investigation.

Speedup relative to FM. We define the *speedup relative* to FM as the ratio of the runing time of FM over that of CPGR. Figure 3d–f illustrate the speedup achieved by CPGR for bipartite graphs with $n = 32$, 64, and 128 vertices in each partition, $m = 819$ to 16 thousand for densities of 0.80, 0.90, and 0.98, and δ values ranging from 0.5 to 1. The running time of CPGR is consistently lower than that of FM, resulting in speedup by CPGR across all cases. This is primarily due to the selection of vertices in the while loop (lines 4–6) in CSA, facilitating greater R by extracting more than one (i.e., γ), δ-cliques in a single iteration, as shown in Figure 2b. The speedup increases notably with higher δ and density. For example, with $n = 32$ and $m = 819$ for density 0.8, CPGR achieves an average speedup of 3.66 for $\delta = 0.5$, increasing to 16.28 for $\delta = 1$. Similarly, with $n = 32$ and $\delta = 1$, CPGR achieves an average speedup ranging from 16.28 for $m = 819$ with density 0.8 to 20.97 for $m = 16$ thousand with density 0.98. It is worth noting that for small graphs, certain combinations of n, m, and δ yield higher speedup values. For instance, with $n = 128$ and $m = 819$ for density 0.8, the speedup increases from 52.81 to 81.97 for $\delta = 0.8$, and then slightly decreases to 60.68 for $\delta = 1$.

5.2 | Results for Large Bipartite Graphs

We present the results obtained from testing CPGR on large bipartite graphs, where each bipartition consists of $n = 2^i$ vertices, with i ranging from 11 to 15, and a number of edges m ranging from approximately 3.36 million to 1.05 billion.

Average reduction in the number of edges (R^{avg}). Figure 4a–c illustrate R^{avg} for CPGR, defined as the percentage of edges removed from the original graph to obtain the restructured graph, that is, $R^{avg} = \frac{m-m^*}{m} \cdot 100$. We observe a consistent trend: for fixed n and δ, R^{avg} improves with increasing density. For example, when $m = 214.75$ million edges at a density of 0.80 and $\delta = 0.5$, R^{avg} is 48.72%, whereas R^{avg} rises to 59% for $m = 1.05$ billion edges at a density of 0.98. This correlation arises because higher density increases the likelihood of finding a large set of common neighbors for the right partition of a δ-clique. Identifying more common neighbors allows CPGR to remove more edges, thereby improving R^{avg}.

However, when the density is held constant, R^{avg} exhibits a non-monotonic behavior depending on δ and n. Consider Figure 4a, for $m = 53.69$ million edges at density 0.80, R^{avg} initially increases from 45.6% at $\delta = 0.5$ to a peak of 50% at $\delta = 0.6$. Further increasing δ to 1, however, leads to a decrease in R^{avg} to 26.5%. As explained before, this is because a larger δ increases the size of the right partition K_q of a δ-clique, which consequently lowers the possibility of finding common neighbors.

Average running time. For a fixed δ and density, the running time of CPGR increases with the number of vertices, consistent with its $O(mn^{\delta})$ complexity. Conversely, for a fixed n and density, the running time decreases as δ increases. For example, at density 0.80 ($m = 214.75$ million), the runtime drops from 176.27 s at $\delta = 0.5$ to 117.8 s at $\delta = 1$. This is because a larger δ limits the search for common neighbors, leading to lower reduction in the number of edges at lower densities and allowing CPGR to skip more vertices. Consequently, the expected increase in runtime with δ is not observed. Notably, even at a high density of 0.98 ($m = 1.05$ billion), the running time still decreases with increasing δ (from 164.83 s at $\delta = 0.5$ to 150.97 s at $\delta = 1$). This is attributed to the extraction of larger δ-cliques at higher densities and δ, which reduces the overall number of iterations in CPGR, thus lowering the runtime.

10970037, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/net.70038 by Wayne State University Library, Wiley Online Library on [22/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

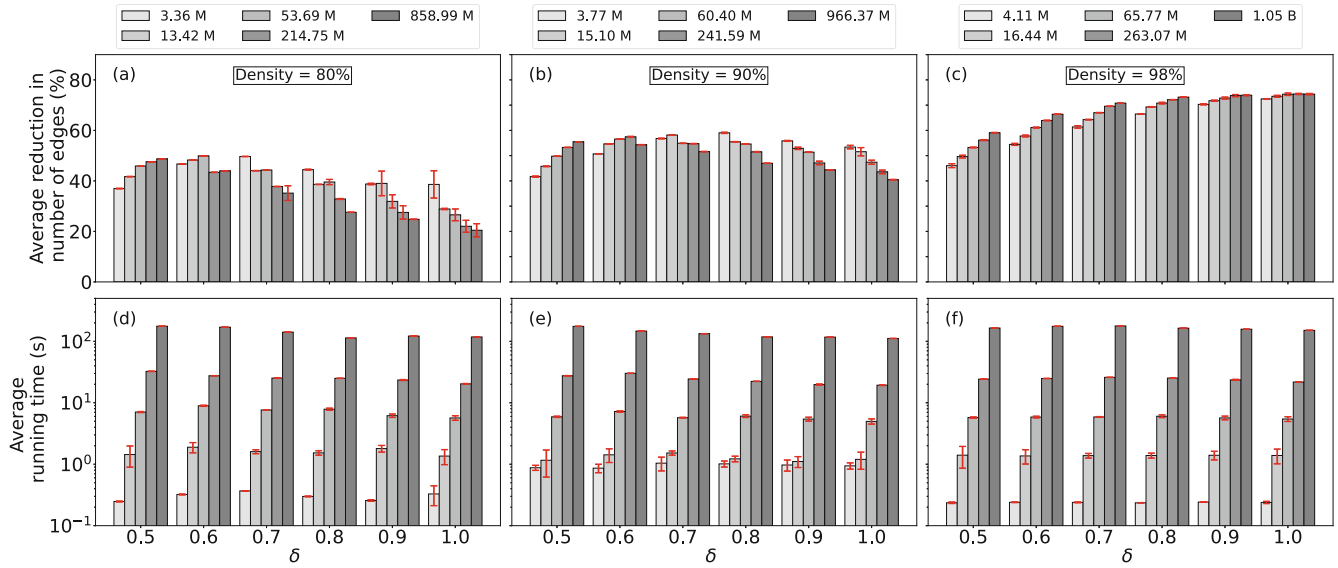


FIGURE 4 | CPGR on Large Graphs: Average reduction in the number of edges R^{avg} ((a), (b), and (c)) and average running time ((d), (e), and (f)) for graphs with 3.36 million to 1.05 billion edges.

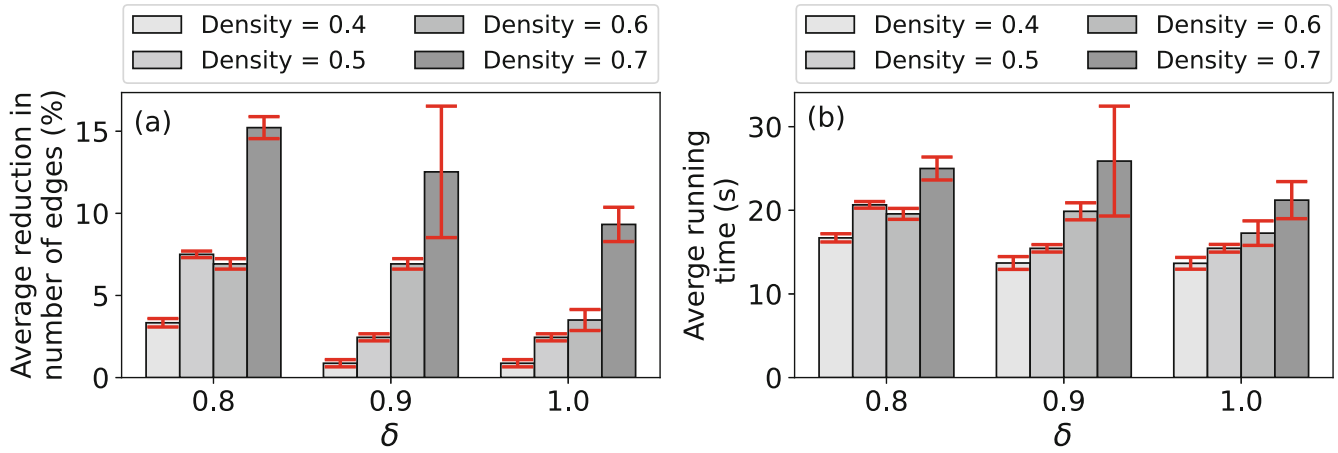


FIGURE 5 | CPGR on General Graphs: (a) average reduction in number of edges R^{avg} and (b) average running time for general graphs with 409 million to 716 million edges.

5.3 | Results for General Graphs

In this section, we analyze CPGR's performance on synthetically generated general graphs with $n = 32\,000$ vertices and number of edges m ranging from approximately 409 million to 716 million as explained in Section 5. We observe that R^{avg} and running time follow the same trend with varying density and δ as seen in the large bipartite graphs.

Average reduction in the number of edges (R^{avg}). Figure 5a shows that for a fixed δ, R^{avg} improves with density. For instance, at $\delta = 0.6$, R^{avg} increases from 12.3% for $m \approx 409$ million at density 0.4 to 28.6% for $m \approx 716$ million at density 0.7, consistent with the increased likelihood of finding common neighbors. Similar to large bipartite graphs, R^{avg} is non-monotonic with δ at a constant density. For $m \approx 716$ million at density 0.7, R^{avg} peaks at 33.3% for $\delta = 0.6$, after initially increasing from 28.6% at $\delta = 0.5$, before dropping to 9.1% at $\delta = 1$.

Average running time. Running time exhibits a non-monotonic trend with increasing δ and density. Larger δ leads to larger right partitions, reducing the likelihood of finding common neighbors in low-density graphs, thus fewer extracted cliques with larger partitions. Conversely, low δ (e.g., 0.5) results in smaller right partitions, increasing the chance of finding common neighbors, leading to higher R and longer runtime, as evident in Figure 5b. The highest runtime (59.96 s) corresponds to the highest R^{avg} (33.3%) at density 0.7 and $\delta = 0.6$, while the runtime reduces to 21.21 s when $\delta = 1$, and R^{avg} drops to 9.1% at the same density.

5.4 | Results for Real-World Graphs

In this section, we analyze CPGR performance on low-density (up to 0.3), symmetric and asymmetric real-world graphs shown in Table 2. Given that $\delta < 0.8$ would yield $\hat{k} = 1$ and thus no

10970037, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/net.70038 by Wayne State University Library, Wiley Online Library on [22/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 2 | Real-world datasets (SuiteSparse matrix collection [16]).

Sr. No	Dataset name	Rows	Columns	Edges	Density
1	beacxc	497	506	50409	0.2
2	beaflw	497	507	53403	0.2
3	cari	400	1200	152800	0.3
4	mbeacxc	496	496	49920	0.2
5	mbeaflw	496	496	49920	0.2

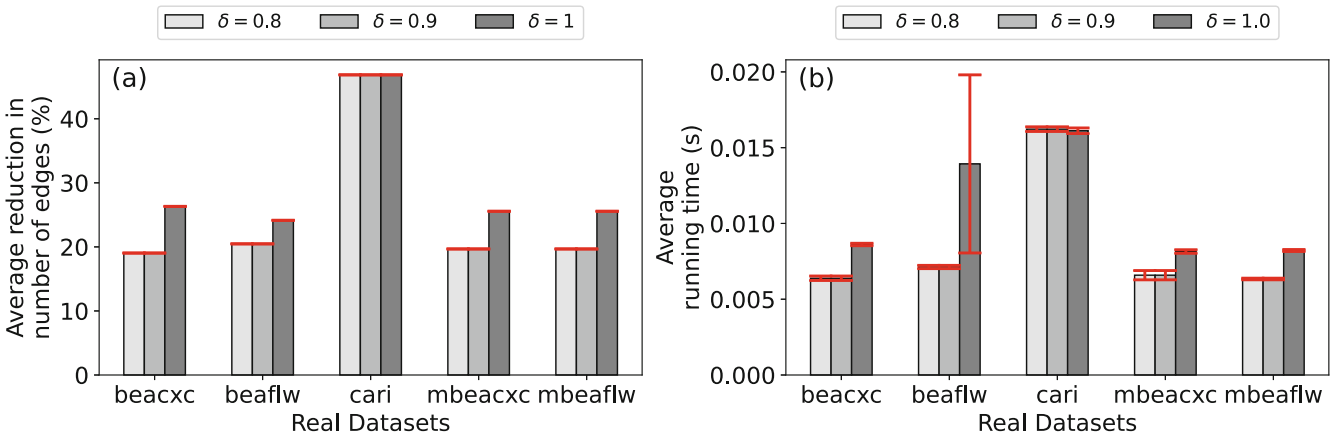


FIGURE 6 | CPGR on Real-world graphs: (a) average reduction in the number of edges (R^{avg}) and (b) average running time.

reduction in the number of edges for these sparse graphs, we tested CPGR with $\delta \in \{0.8, 0.9, 1\}$.

Average reduction in the number of edges (R^{avg}). Figure 6a demonstrates that CPGRachieves R^{avg} ranging from 18.7% to 46.8% even for the relatively low-density real-world graphs examined. Notably, for dataset cari, R^{avg} remains consistent across all tested δ values. This stability is due to low density, which limits the maximum value of $\hat{k}$ to 2, thereby resulting in same the R^{avg} regardless of δ. While for other real-world graphs, R^{avg} increases for $\delta = 1$, this is because, when $\delta = 1$, $\hat{k}$ larger for longer iterations of CPGR (as shown in Figure 2b), resulting in increase in R^{avg}.

Average running time. The running time of CPGR on the real-world datasets is presented in Figure 6b. Similar to R^{avg}, the running time generally remains stable for most datasets, which is due to the same reason of low density limiting $\hat{k}$. However, we observe for the datasets that show increase in R^{avg} at $\delta = 1$, has increase in running time, For example, the average running time for dataset beaflw increases from 0.007 s at $\delta = 0.8$ and 0.9 to 0.014 s at $\delta = 1$. This increase in running time is because at $\delta = 1$, CPGR could extract larger δ-cliques (with $\hat{k} = 2$), leading to higher R^{avg} and thus a longer running time.

5.5 | Speeding-Up Graph Algorithms

In this section, we show the speedup achieved by downstream graph algorithms when we use CPGR algorithm as a preprocessing step across different graph types. Specifically, we present the speedup of the APSP algorithm on real-world datasets and the speedup of Dinitz's algorithm on large synthetic bipartite datasets. Across these algorithms and dataset types, utilizing graphs with reduced number of edges obtained by CPGR as input consistently results in smaller running times.

Speeding-up All-Pairs-Shortest-Path Algorithms. We evaluate the effectiveness of CPGR for speeding up Breadth-First Search (BFS)–based All-Pairs Shortest Path (APSP) computations on unweighted general graphs. Figure 7 shows the running time of APSP on the original graph G and on the restructured graph G^* obtained via CPGR. For each density, the bars represents the average cost of CPGR, APSP(G^*), and APSP(G), measured over 10 runs. To validate correctness, we compare the number of connected vertices in the original and restructured graphs. This validation confirms that vertex-to-vertex reachability is preserved between G and G^*. Moreover, any path discovered by BFS in the restructured graph G^* can be mapped back to a corresponding path in the original graph G by contracting the length-two paths introduced by CPGR for δ-cliques. Therefore, while path lengths in G^* may differ, the APSP results remain correct with respect to the original graph.

Figure 7 shows the results on synthetic graphs with $n = 32\,000$ vertices, varying both graph density and the parameter δ. The results show that, across all tested densities, the combined execution time of CPGR and APSP(G^*) consistently outperforms APSP(G) on large graphs. Although the preprocessing cost increases with density, the reduction in APSP runtime offsets this overhead, yielding speedups of up to 1.74×. These results demonstrate that CPGR remains effective even for large, moderately dense graphs.

10970037, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/net.70038 by Wayne State University Library, Wiley Online Library on [22/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

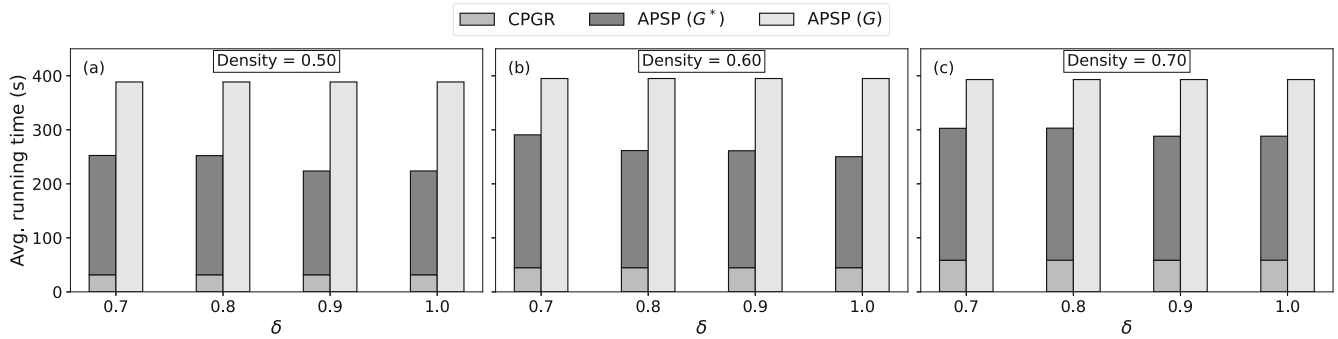


FIGURE 7 | CPGR + APSP(G^*) vs. APSP(G): Average running time of APSP algorithm on the original graph G (labeled APSP(G)) and on the restructured graph G^* (labeled APSP(G^*)), for large graphs with approximately 32 000 vertices and 256 million to 360 million edges corresponding to densities = 0.50, 0.60, and 0.70 and different δ.

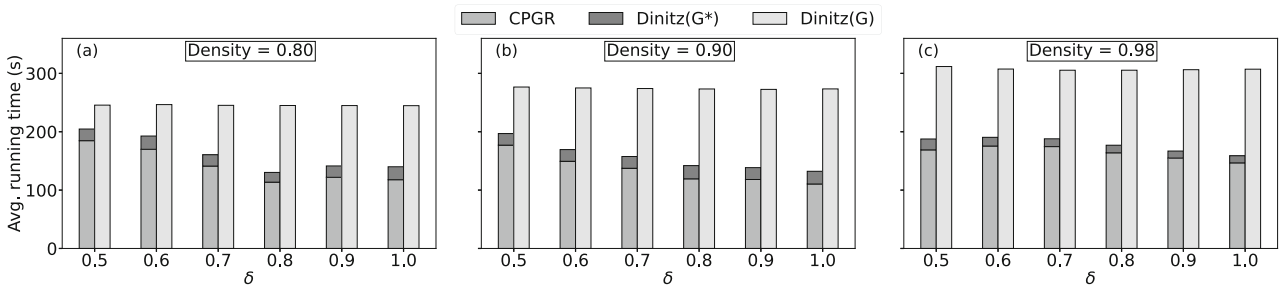


FIGURE 8 | CPGR + Dinitz(G^*) vs. Dinitz(G): Average running time of Dinitz's algorithm on the original bipartite graph G (labeled Dinitz(G)) and on the restructured graph G^* (labeled Dinitz(G^*)), for a large graph with approximately 32 000 vertices in each bipartition and 819.75 million to 1.05 billion edges corresponding to densities = 0.80, 0.90, and 0.98 and different δ.

Speeding-up Matching Algorithms. The restructured graph retains path information, enabling its use as input for other graph algorithms like maximum cardinality matching to accelerate execution. For bipartite matching, we focus on Dinitz's algorithm, which computes a maximum matching via the standard unit-capacity flow formulation. In this formulation, each edge has unit capacity and feasible flows correspond exactly to matching. Under the CPGR restructuring, each edge (u_i, w_j) belonging to an extracted δ-clique is replaced by a two-edge path (u_i, z_q, w_j) in the restructured graph G^*, where both edges have unit capacity. This transformation preserves the set of feasible flows: a flow can traverse (u_i, z_q, w_j) in G^* if and only if the corresponding edge (u_i, w_j) could be used in G, and matching constraints continue to be enforced by the unit-capacity edges incident to vertices in the left and right partitions. Consequently, Dinitz's algorithm computes a maximum flow of the same value on G^* as on G, and the resulting matching can be mapped back to a maximum matching in G in linear time.

Dinitz's algorithm [18], with a runtime of $O(\sqrt{n}m)$ for bipartite matching, benefits from CPGR preprocessing. Given CPGR's reduction in the number of edges $k = \Omega\left(\frac{\delta \log n}{\log(2n^2/m)}\right)$ (matching FM's), the total time becomes $O(\sqrt{n}m/k + mn^{\delta})$, which simplifies to $O(\sqrt{n}m \log_n(\frac{n^2}{m}))$ for $\delta < 1/2$. Notably, this matches the asymptotic bound for maximum cardinality bipartite matching, achieved by both FM and push-relabel methods. Similar speedups are expected with the Hopcroft–Karp algorithm [19]. Better asymptotic bounds (almost linear in m) are achieved by recent bipartite cardinality matching algorithms such as those presented in [20] ($n^{2+o(1)}$-time randomized algorithm) and [21] ($\tilde{O}(m + n^{1.5})$-time randomized algorithm). These algorithms are mainly of theoretical interest, providing the best bounds, and not of practical interest. This is due to the use of sophisticated techniques that require significant implementation effort and they might not even be feasible to implement in practice. Thus, for our experimental analysis we use Dinitz's algorithm as the baseline for comparisons.

Figure 8 compares the running time of Dinitz's algorithm on the original bipartite graph G (n = 32k per partition, m = 214.75 Million to 1.05 Billion edges, densities 0.80–0.98) with the combined time of CPGR preprocessing and Dinitz on the restructured graph G^*. In all cases, the combined time outperforms Dinitz on G, achieving up to a 51.63% reduction (2.07× speedup). On average CPGR + Dinitz (G^*) achieves 1.69× speedup when compared to Dinitz (G). The speedup correlates with the reduction in the number of edges; for example, at density 0.98, the increasing reduction in the number of edges by CPGR with higher m and δ leads to a faster Dinitz on G^*, resulting in up to a 96.07% reduction in Dinitz's runtime compared to G.

6 | Conclusion and Future Work

We proposed a novel Clique Partition based Graph Restructuring (CPGR) algorithm that operates by partitioning the graph into bipartite cliques. The algorithm has a time complexity of $O(mn^{\delta})$. Experimental results demonstrate the effectiveness of

10970037, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/net.70038 by Wayne State University Library, Wiley Online Library on [22/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

CPGR, achieving a reduction in the number of edges by up to 21% greater and execution times up to 105.18 times faster than the baseline FM algorithm. Furthermore, we investigated the impact of using the CPGR-restructured graph as input for a cardinality matching algorithm (Dinitz's algorithm) and an All-Pairs Shortest Path (APSP) algorithm. Experimental results show that for sufficiently large dense graphs, our approach leads to a reduction in the total running time (including CPGR time) of up to 51.63%, resulting in a speedup of 2.07× over the matching algorithm on the original graph. Similarly, for the APSP algorithm, we observed up to 34.78% reduction in total running time, achieving a speedup of up to 1.74×. These speedups for fundamental graph algorithms highlight the practical benefits of our graph restructuring technique, particularly for large-scale real-world applications like route planning and social network analysis where faster processing is critical for efficiency and responsiveness.

Looking ahead, our results indicate that the edge reduction achieved by CPGR is influenced by the interaction between the graph size, the graph density, and the parameter δ. A deeper understanding of these interactions, including the behavior observed for certain dense graphs, remains an open problem. Future work will focus on theoretically characterizing these interactions, developing adaptive strategies for selecting δ, and extending CPGR to dynamic and streaming graph settings.

Author Contributions

Akshar Chavan: conceptualization; investigation; writing – original draft; methodology; validation; visualization; software; formal analysis; data curation; writing – review and editing. **Sanaz Rabinia:** conceptualization; software; validation; methodology; writing – original draft; data curation; investigation. **Daniel Grosu:** conceptualization; investigation; writing – original draft; writing – review and editing; methodology; formal analysis; project administration; supervision; resources; validation. **Marco Brocanelli:** conceptualization; investigation; writing – original draft; writing – review and editing; methodology; supervision; resources; validation.

Funding

The authors have nothing to report.

Data Availability Statement

The implementations of the algorithms presented in this study are openly available in GitHub at https://github.com/srabin1/CPGR-Paper.

References

1. T. Feder and R. Motwani, "Clique Partitions, Graph Compression and Speeding-Up Algorithms," *Journal of Computer and System Sciences* 51, no. 2 (1995): 261–272.

2. M. Hashemi, S. Gong, J. Ni, W. Fan, B. A. Prakash, and W. Jin, "A Comprehensive Survey on Graph Reduction: Sparsification, Coarsening, and Condensation," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI-24*, ed. K. Larson (International Joint Conferences on Artificial Intelligence Organization. Survey Track, 2024), 8058–8066.

3. E. John and I. Safro, "Single- and Multi-Level Network Sparsification by Algebraic Distance," *Journal of Complex Networks* 5, no. 3 (2016): 352–388.

4. D. A. Spielman and S.-H. Teng, "Spectral Sparsification of Graphs," *SIAM Journal on Computing* 40, no. 4 (2011): 981–1025.

5. J. Chen, Y. Saad, and Z. Zhang, "Graph Coarsening: From Scientific Computing to Machine Learning," *SeMA Journal* 79, no. 1 (2022): 187–223.

6. Y. Chen, R. Yao, Y. Yang, and J. Chen, "A Gromov-Wasserstein Geometric View of Spectrum-Preserving Graph Coarsening," in *International Conference on Machine Learning* (PMLR, 2023), 5257–5281.

7. M. Besta and T. Hoefler, "Survey and Taxonomy of Lossless Graph Compression and Space-Efficient Graph Representations," , CoRR, abs/1806.01799, 2018.

8. A. Buluç, H. Meyerhenke, I. Safro, P. Sanders, and C. Schulz, *Recent Advances in Graph Partitioning* (Springer, 2016).

9. M. Epstein, K. Lynch, K. H. Johansson, and R. M. Murray, "Using Hierarchical Decomposition to Speed up Average Consensus," *IFAC Proceedings Volumes* 41, no. 2 (2008): 612–618, 17th IFAC World Congress.

10. Y. Li, W. Li, Y. Tan, F. Liu, Y. Cao, and K. Y. Lee, "Hierarchical Decomposition for Betweenness Centrality Measure of Complex Networks," *Scientific Reports* 7, no. 1 (2017): 46491.

11. M. Bannach, F. A. Marwitz, and T. Tantau, "Faster Graph Algorithms Through DAG Compression," in *41st International Symposium on Theoretical Aspects of Computer Science (STACS 2024), volume 289 of Leibniz International Proceedings in Informatics (LIPIcs)*, ed. O. Beyersdorff, M. M. Kanté, O. Kupferman, and D. Lokshtanov (Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024), 8:1–8:18.

12. G. Buehrer and K. Chellapilla, "A Scalable Pattern Mining Approach to Web Graph Compression With Communities," in *Proceedings of International Conference On Web Search and Data Mining*, WSDM '08 (ACM, 2008), 95–106.

13. K. Chakrabarti, E. Keogh, S. Mehrotra, and M. Pazzani, "Locally Adaptive Dimensionality Reduction for Indexing Large Time Series Databases," *ACM Transactions on Database Systems (TODS)* 27, no. 2 (2002): 188–228.

14. L. Dhulipala, I. Kabiljo, B. Karrer, G. Ottaviano, S. Pupyrev, and A. Shalita, "Compressing Graphs and Indexes With Recursive Graph Bisection," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16 (ACM, 2016), 1535–1544.

15. W. Fan, Y. Li, M. Liu, and C. Lu, "Making Graphs Compact by Lossless Contraction," in *Proceedings of the 2021 International Conference on Management of Data*, SIGMOD '21 (ACM, 2021), 472–484.

16. T. A. Davis and Y. Hu, "The University of Florida Sparse Matrix Collection," *ACM Transactions on Mathematical Software* 38, no. 1 (2011): 1–25.

17. "Implementation of the Algorithms is Available on Github," https://github.com/srabin1/CPGR-Paper.

18. Y. Dinitz, "Algorithm for Solution of a Problem of Maximum Flow in a Network With Power Estimation," *Doklady Akademii Nauk SSSR* 11 (1970): 1277–1280.

19. J. E. Hopcroft and R. M. Karp, "An $n^{5/2}$ Algorithm for Maximum Matchings in Bipartite Graphs," *SIAM Journal on Computing* 2, no. 4 (1973): 225–231.

20. J. Chuzhoy and S. Khanna, "Maximum Bipartite Matching in $n^{2+o(1)}$ Time via a Combinatorial Algorithm," in *Proceedings of the 56th Annual ACM Symposium on Theory of Computing*, STOC 2024 (Association for Computing Machinery, 2024), 83–94.

21. J. van den Brand, Y. T. Lee, D. Nanongkai, et al., "Bipartite Matching in Nearly-Linear Time on Moderately Dense Graphs," in *Proceedings of*

10970037, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/net.70038 by Wayne State University Library, Wiley Online Library on [22/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

61st IEEE Annual Symposium on Foundations of Computer Science (IEEE, 2020), 919–930.

Appendix A

FM Algorithm

In Algorithm 3 we give a high-level description of the FM algorithm. The FM algorithm first constructs *neighborhood trees* (Figure A1) for each vertex u_i in U, which are labeled binary trees of depth r, whose nodes are labeled by a bit string ω, where $0 \le |\omega| \le r$. The root node of a neighborhood tree contains all the vertices in W and is labeled by the empty string, ϵ. The bit string ω of a node is obtained by starting at the root and following the path in the tree up to the node and concatenating a 0 or a 1 to ω each time the path visits the left, or the right child, respectively. The nodes at level i in the tree correspond to a partition of W into sets of size $|W|/2^i$. Thus, each leaf node corresponds to a vertex in W. Each node in the neighborhood tree of u_i stores $d_{i,\omega}$, the number of neighbors of u_i in the set W_ω of vertices of W that are associated with the node with label ω, that is, $d_{i,\omega} = |N(u_i) \cap W_\omega|$, where $N(u_i)$ is the set of the neighbors of u_i. In Figure A1, we show the neighborhood tree of vertex u_2 for the bipartite graph with $|U| = |W| = n = 8$ given in Figure 2. Vertex w_3 is a leaf and would have $\omega = 010$ in all neighborhood trees of the graph.

The algorithm computes $\hat{k}$ and then, in line 5, it performs $\hat{k}$ iterations (indexed by t) to select vertices for the right partition of clique C_q. In each iteration it selects vertices at leaf nodes of the neighborhood trees by following a path based on $d_{i,\omega}$ of child nodes and the number of distinct ordered subsets of the neighborhood tree at each level. In order to select a path at each level, the algorithm calculates c_0 and c_1 as follows:

$$c_j = C\left(U_t, \mathscr{C}_{t,\omega\cdot j}\right) = \sum_{i:u_i \in U_t} d_{i,\omega\cdot j} \cdot \left(d_i - 1\right)^{[k-t]},\ j = 0, 1. \qquad \text{(A1)}$$

c_j counts the number of ordered sets in $\mathscr{C}_{t,\omega}$ whose elements are adjacent to u_i, where $\mathscr{C}_{t,\omega}$ denotes the collection of all ordered sets K of W of size k in iteration t, with $t \le k$. In each iteration, a vertex in W is added to the right partition of the clique. If $|N(u_i)| = d_i$, then the

ALGORITHM 3 | FM Algorithm: High-level description.

Input: $G(U, W, E)$: Bipartite graph;

1: Initialize: clique index, $q \leftarrow 0$; no. of vertices $n \leftarrow |U|$; and no. of remaining edges $\hat{m} \leftarrow |E|$.
2: Build neighborhood trees for vertices $u_i \in U$.
3: **while** $\hat{m} \ge n^{2-\delta}$ **do**
4: $\quad \hat{k} \leftarrow \left\lfloor \frac{\delta \log n}{\log(2n^2/\hat{m})} \right\rfloor$
5: $\quad$ Starting with $t = 1$ and $U_t = U$, iterate while $t \le \hat{k}$ to select $\hat{k}$ vertices based on c_0 and c_1 (calculated as in Equation A.1) to form the ordered set $K_q = \{y_1, y_2, \ldots, y_t\} \subseteq W$. At each stage t, obtain set $U_{t+1} = \left\{u \in U_t \mid y_1, \ldots, y_t \in N(u)\right\}$ which omits vertices from U that are not adjacent to K_q. And update the corresponding neighborhood trees such that $d_{i,\omega}$ of the nodes along the path from root to the selected vertex is decreased by 1 for each neighborhood tree corresponding to U_t.
6: $\quad$ Form the δ-clique $C_q = (U_q, K_q)$, where, $U_q = U_t$.
7: $\quad$ Update G by removing the edges associated with clique C_q and update $\hat{m} \leftarrow |E|$.
8: $\quad q \leftarrow q + 1$
9: For each clique in $\mathcal{C} = \{C_1, \ldots, C_q\}$ add edges from all the vertices in the left partition U_q to an additional vertex z_q, and from z_q to all the vertices in the right partition K_q. This would restructure the graph by replacing $|U_q| \times |K_q|$ edges with $|U_q| + |K_q|$ edges.
10: The remaining edges in G are trivial cliques and are added in the restructured graph G^*.
11: **Output:** G^*, the restructured graph of G.

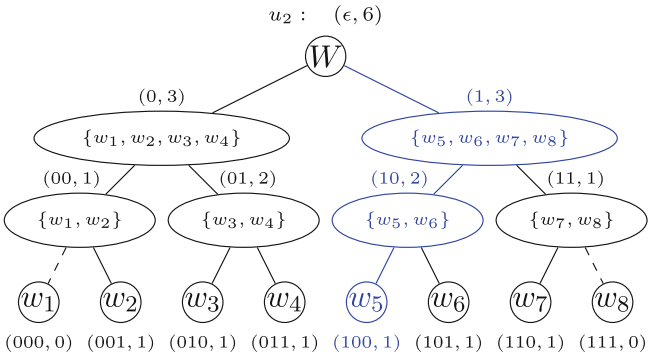


FIGURE A1 | Neighborhood tree of vertex $u_2 \in U$ that shows the path ω taken from root to the vertex $w_j \in W$ at the leaf and number of edges $d_{u_2,\omega}$ for each node using the tuple $(\omega, d_{u_2,\omega})$.

number of distinct ordered subsets of $N(u_i)$ of size k is $d_i^{[k]}$, where $d_i^{[k]} = d_i(d_i - 1)(d_i - 2)\cdots(d_i - k + 1)$. Based on c_j's, the FM algorithm chooses either the left node (if $c_0 \ge c_1$) or the right node (if $c_0 < c_1$). When the algorithm reaches a leaf node, it selects the corresponding vertex in W, updates the set of common neighbors (U_t), and decreases $d_{i,\omega}$ of each node along the path to the selected vertex by 1, in neighborhood trees corresponding to U_t. Thus, it guarantees the selection of a new vertex in the following iterations. The algorithm continues this process until it extracts $\hat{k}$ unique vertices to form the set K_q, the right partition of the qth δ-clique C_q.

After forming set K_q, the FM algorithm proceeds to identify the common neighbors of the vertices that are part of set K_q in order to construct U_q, which is the left partition of the qth δ-clique C_q. The pair (K_q, U_q) forms a δ-clique C_q. Subsequently, the algorithm updates $\hat{m}$, the number of remaining edges in the graph after extracting each δ-clique. This update is done by subtracting the product of the sizes of K_q and U_q from $\hat{m}$, that is, $|U_q| \times |K_q|$. The algorithm then updates $\hat{k}$ considering the updated value of $\hat{m}$ and repeats the procedure to extract more δ-cliques until no more δ-cliques can be found (determined by the condition of the while loop). The edges that are not removed in this process form trivial cliques.

The FM algorithm restructures the graph by adding a new vertex set Z to the bipartite graph, thus converting it into a tripartite graph (Figure 2a), where each vertex in Z corresponds to one clique extracted by the algorithm. Each vertex of the left and right partitions of δ-clique C_q is then connected via an edge to a new vertex $z_q \in Z$, thus forming a tripartite graph. This decreases the number of edges from $|U_q| \times |K_q|$ to $|U_q| + |K_q|$, thus reducing the number of edges in the graph. The restructured graph obtained by the algorithm preserves the path information of the original graph. The running time of the FM algorithm is $O(mn^\delta \log^2 n)$. The FM algorithm can be *extended to the case of non-bipartite graphs*, where it restructures a graph in time $O(mn^\delta \log^2 n)$, as shown by Feder and Motwani [1].

Example: CPGR Execution

We now show how CPGR works on a bipartite graph G with partitions U and W, $n = |U| = |W| = 8$, and $|E| = 54$, shown in Figure A2a. CPGR (Algorithm 1), first initializes $q = 0$, $n = |W| = |U| = 8$, $\hat{m} = |E| = 54$, $C = \emptyset$, and $d_w = [0]_n$. It then calculates the degree of each vertex d_{w_j} in lines 6–8, as shown in Table A1. It calculates $\hat{k}(n, m, \delta) = 2$ in line 9. Thus, the condition for the while loop in line 10 is met and CPGR calls CSA in line 11. With the given inputs, CSA sorts d_w in non-increasing order. Let $\{w_4, w_2, w_3, w_5, w_6, w_1, w_7, w_8\}$ be the non-increasing order with corresponding degrees given as $d_{w_{\pi(j)}}$ in Table A1.

In line 5, CSA forms set $\mathcal{K}$, where $\mathcal{K} = \{w_4, w_2, w_3, w_5, w_6\}$ has all vertices whose degrees are greater than or equal to $d_{w_{\pi(\hat{k})}} = 7$. This results in $\gamma = 2$ in line 7, therefore CSA extracts two cliques in lines 8–13. Since q is initialized to 0 in CPGR, the for loop in line 8 iterates two times

10970037, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/net.70038 by Wayne State University Library, Wiley Online Library on [22/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

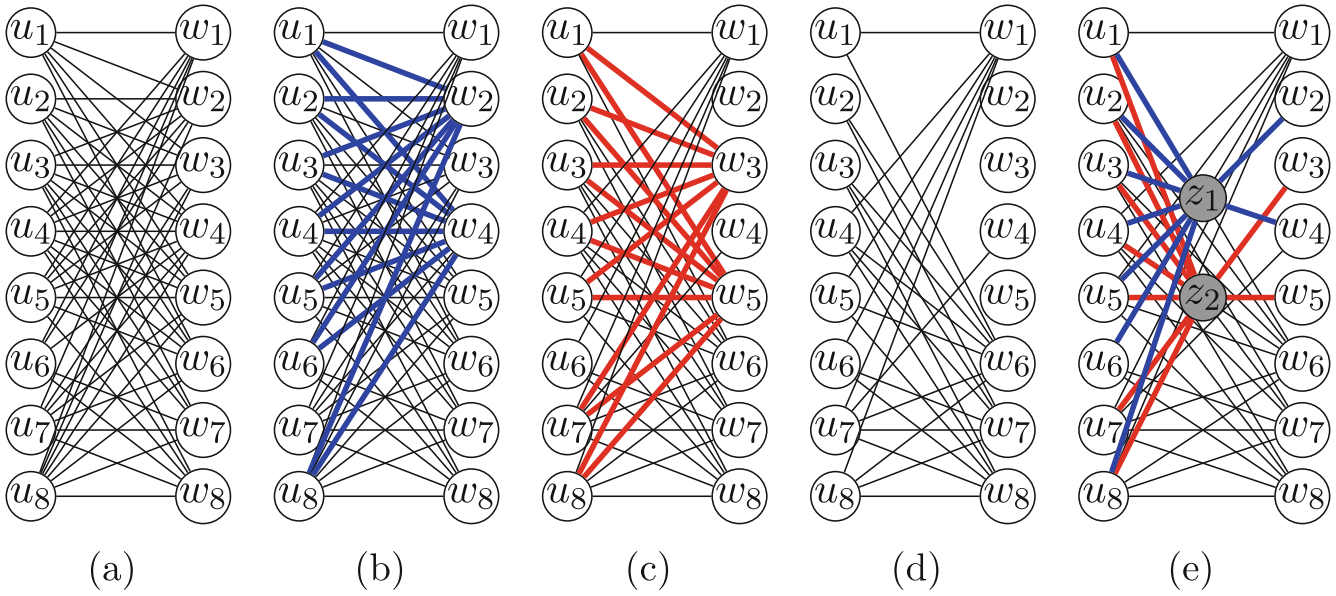


FIGURE A2 | Clique partitioning: (a) given bipartite graph $G(U, W)$; (b) extracted clique C_1; (c) extracted clique C_2; (d) graph with trivial cliques; and (e) restructured tripartite graph $G^*(U, Z, W)$.

TABLE A1 | Degrees of vertices in partition W at different steps of CPGR.

CPGR step	j	1	2	3	4	5	6	7	8
Initialization	d_{w_j}	6	7	7	8	7	7	6	6
Sorted d_{w_j}	$d_{w_{\pi(j)}}$	8	7	7	7	7	6	6	6
After 1st iteration	d_{w_j}	6	0	0	1	0	7	6	6

for $c = 1$ and 2. In line 9, CSA forms $K_1 = \{w_4, w_2\}$, and in line 10, it forms the left partition U_{K_1} of bipartite clique C_1, by selecting the common neighbors of the vertices in K_1. For example, for set $K_1 = \{w_4, w_2\}$, $U_{K_1} = U - \{u_7\}$ as w_4 and w_2 have $\{u_1, u_2, u_3, u_4, u_5, u_6, u_8\}$ as common neighbors. In line 12, CSA updates the adjacency matrix A by removing the edges of bipartite clique C_1 from the original bipartite graph G and finally in line 13, it updates all d_w by subtracting $|U_{K_1}| = 7$ from both d_{w_4} and d_{w_2}, thus $d_{w_4} = 1$ and $d_{w_2} = 0$. Similarly, when $c = 2$, $K_2 = \{w_3, w_5\}$ and $U_{K_2} = U - \{u_6\}$. CSA updates the adjacency matrix A and d_w such that $d_{w_3} = d_{w_5} = 0$. Therefore, in the first execution, CSA, forms two bipartite cliques and removes a total of $(7 \times 2) + (7 \times 2) = 28$ edges from G. The updated degrees of the vertices are shown in Table A1. It then forms the set of bipartite cliques extracted in the current execution in line 14, $\mathscr{C} = \{C_1, C_2\}$. The two cliques are shown in Figures A2b and A2c.

CSA returns $(\mathscr{C}, \hat{A}, \hat{d}_w)$ and then CPGR updates the set $\mathscr{C} = \{C_1, C_2\}$, d_w, the adjacency matrix A, and the number of edges $\hat{m} = 54 - 28 = 26$, in lines 12–15. In line 16, CPGR updates $q = 2$ and finally, in line 17 it updates $\hat{k}$ according to the new value of $\hat{m}$ in line 15 which results in $\hat{k} = 1$. Since $\hat{k} = 1$, it means CPGR extracts only trivial bipartite cliques (shown in Figure A2d) which do not contribute to reduction in the number of edges in the restructured graph. Thus, it does not meet the condition in the while loop (line 6) and CPGR terminates.

CPGR then restructures the graph by adding two vertices, z_1, and z_2, corresponding to the two cliques C_1 and C_2, and adds the corresponding edges to form the tripartite graph. The edges in the given graph $G(U, W, E)$, that are not part of any δ-clique are connected directly as shown in Figure A2e.

10970037, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/net.70038 by Wayne State University Library, Wiley Online Library on [22/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License